BÁO CÁO THỰC HÀNH

LAB01 - EMBEDDED LINUX SYSTEMS

Họ và tên: Doan Truong Son

Mã sinh viên: SE200908

Môn học: LOS201 - Embedded Linux Systems

Bài thực hành: LAB01

1. MỤC TIÊU BÀI THỰC HÀNH

Bài thực hành LAB01 giúp sinh viên làm quen với môi trường Embedded Linux, tìm hiểu quá trình xây dựng hệ điều hành Linux nhúng và khởi động hệ thống trên máy ảo QEMU.

Các mục tiêu chính:

- Tìm hiểu và biên dịch Linux Kernel 5.15.
- Tìm hiểu và biên dịch BusyBox.
- Xây dựng Root Filesystem.
- Tạo initramfs cho hệ thống Linux.
- Khởi động hệ thống bằng QEMU với board vexpress-a9.

2. MÔI TRƯỜNG THỰC HIỆN

Thành phần	**Phiên bản / cấu hình**
Hệ điều hành	Ubuntu
Linux Kernel	5.15
U-	2022.04

Boot	
BusyBox	1.36.1
QEMU Machine	vexpress-a9
CPU	Cortex-A9
Cross Compiler	arm-linux-gnueabihf

Lưu ý: Đề bài yêu cầu BusyBox phiên bản 1.35.0, tuy nhiên phiên bản nguồn được sử dụng trong quá trình thực hiện là 1.36.1.

3. QUY TRÌNH THỰC HIỆN

3.1. Biên dịch Linux Kernel

Linux Kernel là thành phần cốt lõi của hệ điều hành, chịu trách nhiệm quản lý bộ nhớ, CPU, thiết bị và các tiến trình. Trong bài thực hành, kernel phiên bản 5.15 được sử dụng.

3.2. Biên dịch BusyBox

BusyBox cung cấp nhiều lệnh Linux cơ bản trong một

chương trình duy nhất. BusyBox được sử dụng để cung cấp môi trường dòng lệnh cho hệ thống Linux nhúng.

3.3. Xây dựng Root Filesystem

Root Filesystem chứa các thư mục và chương trình cần thiết để hệ thống Linux hoạt động.

```
rootfs/
├── bin/
├── dev/
├── etc/
├── proc/
├── sbin/
├── sys/
```

```
├── tmp/




└── usr/
```

Trong root filesystem, init được liên kết tới sbin/init và linuxrc được liên kết tới bin/busybox.

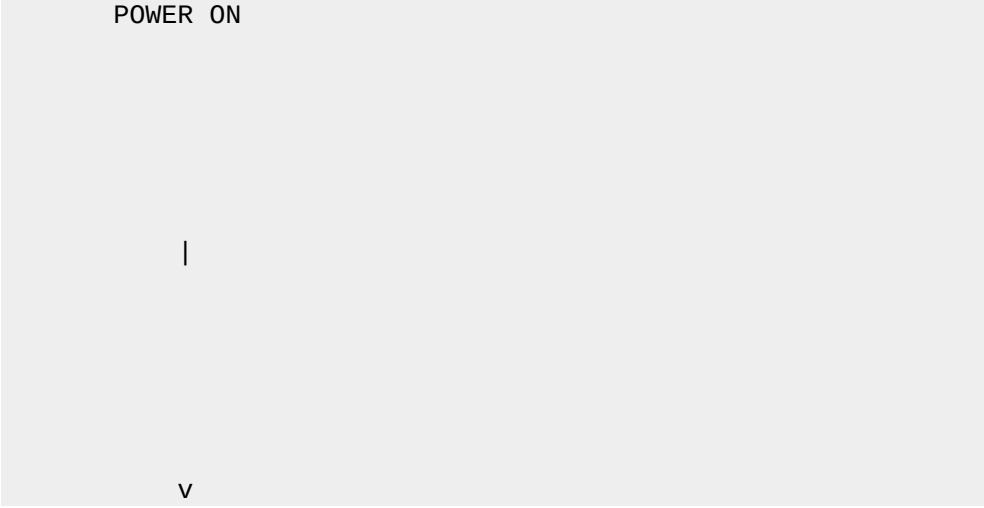
3.4. Tạo initramfs

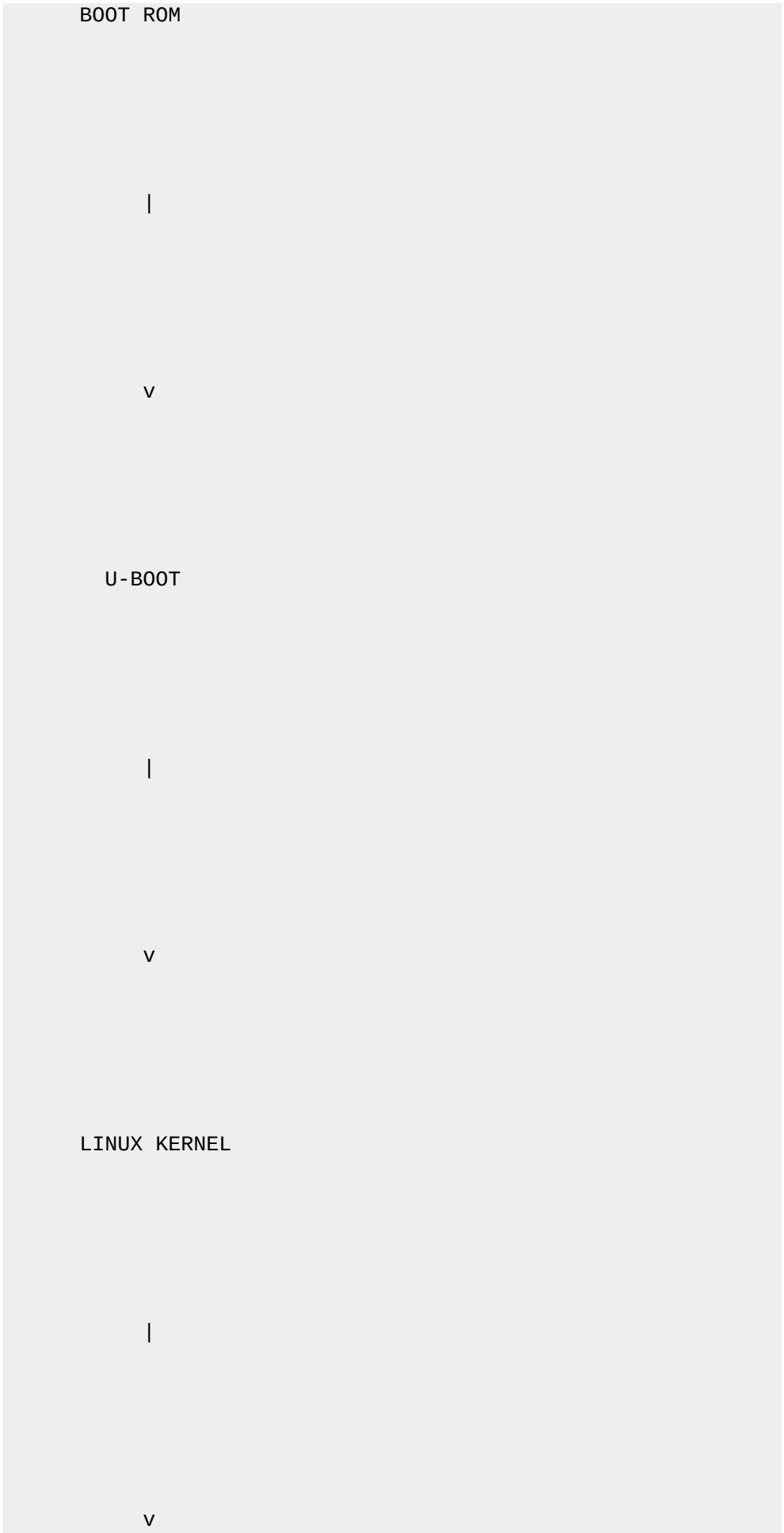
Initramfs là filesystem ban đầu được kernel nạp vào bộ nhớ khi khởi động. Nó cung cấp các chương trình và thư mục cần thiết để hệ thống bắt đầu hoạt động.

3.5. Khởi động bằng QEMU

QEMU được sử dụng để mô phỏng board vexpress-a9 và kiểm tra khả năng khởi động của Linux Kernel cùng initramfs.

4. SƠ ĐỒ QUÁ TRÌNH KHỞI ĐỘNG




BOOT ROM
U-BOOT
LINUX KERNEL

```
      /sbin/init
          |
          v
      BUSYBOX SHELL
```

5. KẾT QUẢ THỰC HIỆN

Các file đầu ra được chuẩn bị gồm:

- zImage: Linux Kernel đã biên dịch.
- vexpress-v2p-ca9.dtb: Device Tree Blob.
- initramfs.cpio.gz: Root filesystem đã đóng gói.
- kernel.config: Cấu hình Linux Kernel.
- busybox.config: Cấu hình BusyBox.

Người thực hiện xác nhận hệ thống đã khởi động thành công trên QEMU với board vexpress-a9.

Các lệnh có thể sử dụng để kiểm tra hệ thống:

```
uname -a
```

```
cat /proc/cpuinfo
```

```
cat /proc/meminfo




ls -l /dev




mount




ps
```

Lưu ý: Các lệnh trên là lệnh kiểm tra tham khảo. Báo cáo không khẳng định kết quả đầu ra cụ thể khi chưa có log console được lưu lại.

6. TRẢ LỜI CÂU HỎI

Câu 1. So sánh arm-linux-gnueabi và arm-linux-gnueabihf

arm-linux-gnueabi thường sử dụng soft-float ABI, trong khi arm-linux-gnueabihf sử dụng hard-float ABI. Hard-float cho phép sử dụng các thanh ghi FPU để truyền tham số dấu phẩy động theo ABI tương ứng.

Cortex-A9 hỗ trợ phần cứng dấu phẩy động VFP. Vì vậy, hard-float có thể tận dụng khả năng xử lý dấu phẩy động của CPU khi userspace và các thư viện được biên dịch tương thích với nhau.

Cần lưu ý kernel không sử dụng userspace floating-point ABI theo cùng cách các ứng dụng sử dụng. Do đó, việc kernel được biên dịch bằng gnueabi không đồng nghĩa hệ thống chắc chắn không khởi động được. Vấn đề tương thích chủ yếu nằm ở ứng dụng và thư viện userspace.

Câu 2. Vì sao RAM của vexpress-a9 bắt đầu tại 0x60000000?

Mỗi board có memory map riêng do phần cứng quy định. Vexpress-a9 ánh xạ RAM bắt đầu tại địa chỉ 0x60000000, khác với một số board như Versatile PB.

Do đó, địa chỉ nạp kernel, DTB và initramfs trong U-Boot phải phù hợp với memory map của board. Nếu sử dụng sai địa chỉ, dữ liệu có thể bị ghi sai vùng bộ nhớ, chồng lấn hoặc khiến kernel không thể khởi động.

Câu 3. Giải mã MIDR = 0x410fc090

Trường	Giá trị	Ý nghĩa
Implementer	0x41	ARM
Variant	0x0	Variant 0

nt		
Architecture	0xF	Giá trị đặc biệt, cần đối chiếu ID register
PartNumber	0xC09	Cortex-A9
Revision	0x0	Revision 0

Từ các trường Implementer và Part Number có thể xác định CPU là ARM Cortex-A9.

Câu 4. Vì sao BusyBox thường được liên kết tĩnh?

Liên kết tĩnh giúp BusyBox hoạt động trong root filesystem tối giản mà không cần nạp các thư viện chia sẻ bên ngoài. Điều này phù hợp với initramfs vì môi trường khởi động cần hạn chế các phụ thuộc.

Nếu liên kết động, chương trình có thể cần các thư viện như:

- libc.so.6
- libm.so.6
- libpthread.so.0
- libgcc_s.so.1
- ld-linux-armhf.so.3

Danh sách thực tế phụ thuộc vào cấu hình biên dịch và cần được kiểm tra bằng readelf hoặc ldd trên binary.

Câu 5. Sự khác nhau giữa SMP và UP

UP (Uniprocessor) sử dụng một CPU để thực thi hệ điều hành. SMP (Symmetric Multiprocessing) cho phép nhiều CPU cùng tham gia xử lý công việc.

Khi chạy với -smp 2, kernel cần phát hiện CPU phụ, khởi tạo cấu trúc dữ liệu per-CPU, khởi động CPU phụ và đưa CPU đó vào trạng thái online.

Ví dụ các thông báo có thể xuất hiện trong log SMP:

```
SMP: Total of 2 processors activated




CPU1: Booted secondary processor
```

Đây là ví dụ minh họa, không phải log được thu thập từ phiên chạy thực tế của bài thực hành.

7. KHÓ KHĂN VÀ CÁCH KHẮC PHỤC

Khó khăn 1: Sao chép device node

Khi sao chép root filesystem, các thiết bị trong thư mục /dev như console, null và random có thể không được sao chép bằng lệnh cp thông thường.

Cách khắc phục: sử dụng quyền quản trị và các công cụ bảo toàn metadata như cp -a hoặc tar.

Khó khăn 2: Tổ chức thư mục nộp bài

Các file kernel, cấu hình, root filesystem và báo cáo cần được sắp xếp theo đúng cấu trúc yêu cầu.

Cách khắc phục: tạo các thư mục output, configs, rootfs và bao_cao để quản lý từng thành phần.

Khó khăn 3: Khác biệt phiên bản BusyBox

Phiên bản BusyBox sử dụng là 1.36.1 trong khi yêu cầu bài thực hành ghi phiên bản 1.35.0. Cần ghi nhận sự khác biệt và build lại đúng phiên bản nếu giảng viên yêu cầu.

8. KẾT LUẬN

Qua bài thực hành LAB01, sinh viên đã tìm hiểu quy trình xây dựng một hệ thống Embedded Linux gồm Linux Kernel, BusyBox và Root Filesystem. Các thành phần đầu ra đã được chuẩn bị và hệ thống được xác nhận khởi động thành công trên QEMU với board vexpress-a9.

Bài thực hành giúp hiểu rõ hơn quá trình khởi động Linux, vai trò của kernel, initramfs, BusyBox và môi trường mô phỏng

QEMU.

9. TÀI LIỆU THAM KHẢO

1. Tài liệu hướng dẫn LAB01 - Embedded Linux Systems.
2. Linux Kernel Documentation.
3. BusyBox Documentation.
4. QEMU Documentation.
5. U-Boot Documentation.